

SOSU AI Service Layer SSOT

SOSU AI Service Layer SSOT

1. 시스템 경계
2. 화면과 AI 데이터 흐름
3. AI에 들어가는 input의 다섯 분류
4. Q1-Q20 입력 계약 - 화면부터 EvidencePack까지
5. Stable code dictionary
6. Transport request와 내부 EvidencePack은 같은 객체가 아니다
7. 자연어 처리 원칙 - Python은 의미를 빼앗지 않는다
8. 이미지 경로 - 저장소 없이 처리하는 기본형
9. 메인 문진은 Gemini 2단 호출을 기본으로 한다
10. EvidencePackV1 - 최종 synthesis 입력
11. InferencePolicyV1 - “어텐션”을 section routing으로 통제
12. Call B - Diagnosis synthesis prompt composition
13. 결과 계약 - UI 제목이 아니라 데이터만 반환
14. Python validation - schema valid만으로 성공하지 않는다
15. Token/length 정책
16. 처방전 AI 계약
17. Data plane과 Control plane을 분리한다
18. API surface
19. Error contract
20. Monorepo와 별도 에이전트 세션 경계
21. Storybook과 AI contract test의 연결
22. End-to-end 예상 시나리오
23. 구현 acceptance gate
24. 최종 데이터 계층 요약

SOSU AI Service Layer SSOT

기준일: 2026-09-13

대상: apps/web와 services/ai를 모노레포 안에서 별도 로컬 워크스페이스/에이전트 세션으로 독립 구현하기 위한 AI 서비스 계약

목적: 프론트 코드를 보지 않는 AI 에이전트도 화면 입력, 정규화, 추론, 멀티모달 처리, structured output, 오류 반환을 완결적으로 구현할 수 있게 한다.

핵심 원칙: 사용자의 자연어는 최대한 원문으로 보존하고, 선택형 응답만 deterministic stable code로 변환한다. 이미지는 문진 prior와 분리해 독립 관측한 뒤, section별 evidence routing에 따라 최종 진단에 합성한다. React는 Gemini의 텍스트를 파싱하지 않고 검증된 public ViewModel만 렌더링한다.

1. 시스템 경계

SOSU의 메인 문진 진단은 단순한 “이미지 분석 API”가 아니다. 하나의 진단 request에는 다음 네 가지 정보원이 동시에 존재한다.

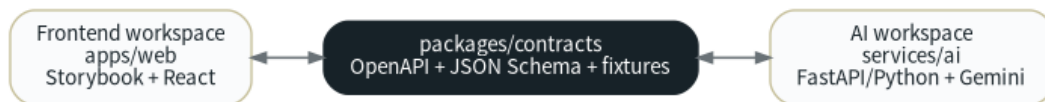
1. **자유 자연어**: 사용자가 직접 쓰는 브랜드, 제품, 차별점, 고객, 릴스 방향 문장.
2. **사전정의 categorical 선택**: 디자인, 희소성, 예뻐서, 설렘, 조회수 문제, 얼굴 공개 가능 여부 등.
3. **멀티모달 evidence**: 제품 이미지 또는 처방전의 Instagram Insight screenshot.
4. **앱 제어값**: 다음, 진단, 메일 전송, reset, 탭 이동 등. 이것은 AI evidence가 아니며 Gemini로 보내지 않는다.

AI service의 책임은 다음과 같다.

- request transport validation
- 자유입력 원문 정규화
- categorical stable code 검증
- hard constraint derivation
- image normalization
- image-only observation call
- EvidencePackV1 조립
- 결과 section별 InferencePolicyV1 적용
- Gemini structured-output synthesis
- Pydantic/JSON Schema 검증
- semantic validation 및 필요 시 repair 1회
- React가 바로 그릴 수 있는 public ViewModel 반환

프론트의 책임은 다음으로 제한한다.

- UI state 및 Storybook 상태
- 선택 label과 stable code의 연결
- 사용자 입력 수집
- 브라우저 이미지 1차 압축
- multipart request 생성
- public ViewModel 렌더링
- 오류 code에 맞는 UX 표시



워크스페이스 경계

2. 화면과 AI 데이터 흐름

2.1 메인 문진 화면

현재 화면 구조는 세 단계의 질문 + 이미지 업로드 + 진단 결과로 구성된다.

화면	질문 범위	AI 관점
Survey 1	Q1-Q7	제품/브랜드 기본 사실 + 핵심 매력 + 차별점
Survey 2	Q8-Q13	고객 의도 + 감정 + 브랜드 이미지/금지 이미지/모티프
Survey 3	Q14-Q18, Q20	릴스 메시지 + 제작과정 + pain + 현재 포맷 + 제작자 제약
Upload	제품 이미지 0..N	문진 prior와 별개인 visual evidence
Result	고정 섹션	DiagnosisReportV1을 deterministic하게 렌더링

SOSU

문진표

처방전

소규모 수공예 브랜드 문진표

브랜드 기본 정보

1. 브랜드 이름은 무엇인가요?
입력

2. 어떤 제품을 만들고 있나요?
입력

3. 가장 자신 있는 제품은 무엇인가요?
입력

4. 주로 어떤 소재를 사용하나요?
입력

5. 이 소재를 사용하는 이유는 무엇인가요?
입력

6. 내 제품의 가장 큰 매력은 무엇인가요?
디자인 색감 디테일
질감 소재 제작 과정

다음

Survey 1 · Q1-7

SOSU

문진표

처방전

소규모 수공예 브랜드 문진표

고객과 브랜드 이미지

8. 내 제품을 가장 좋아할 사람은 누구인가요?
입력

9. 고객은 왜 이 제품을 사고 싶어 할까요?
예배서 특별해서
나를 위해 선물하려고
추억 기타

10. 고객이 어떤 기분을 느꼈으면 좋겠나요?
디자인 색감 디테일
질감 소재 제작 과정

11. 우리 브랜드를 표현하는 단어는?
디자인 색감 디테일
질감 소재 제작 과정

12. 피하고 싶은 이미지는 무엇인가요?
입력

다음

Survey 2 · Q8-13

SOSU

문진표

처방전

소규모 수공예 브랜드 문진표

릴스의 방향성

14. 제품에서 가장 어렵고 싶은 것은?
입력

15. 신기한 제작 과정이 있나요?
입력

16. 인스타그램에서 가장 어려운 점은?
예배서 특별해서
나를 위해 선물하려고
추억 기타

17. 지금 어떤 릴스를 주로 올리고 있나요?
예배서 특별해서
나를 위해 선물하려고
추억 기타

18. 얼굴과 목소리 공개가 가능 한가요?
예배서 특별해서
나를 위해 선물하려고

다음

Survey 3 · Q14-20

문진 3단계 화면



멀티모달 입력 화면

2.2 처방전 화면

처방전은 메인 문진과 inference source가 다르다.

- Reel URL: provenance/context identifier. v1에서는 해당 URL을 크롤링하거나 Instagram 콘텐츠를 자동 fetch하는 근거로 사용하지 않는다.
- Insight screenshot 정확히 3장: 실제 분석 evidence.
- 결과: prescription, next_actions, tone의 고정 구조.

SOSU

문진표

처방전

릴스 진단서

브랜드 요약

브랜드 이름 · 주 소재 · 대표 제품 · 핵심 키워드

추천 포지셔닝

브랜드가 릴스에서 어떤 인상을 선점해야 하는지 요약합니다.

핵심 강점

문진과 이미지에서 드러난 강점을 콘텐츠 관점으로 정리합니다.

메인 타겟

가장 먼저 반응할 고객과 메시지 방향을 설명합니다.

톤앤매너

색감, 자막, 화면 리듬과 일투의 권장 방향을 제시합니다.

추천 릴스 구조

0~3초 → 3~10초 → 10~20초 → 20~27초 → 27~30초

실행 체크리스트

다음 촬영 전에 바로 적용할 액션을 우선순위로 정리합니다.

진단서 받기

Diagnosis Report · fixed section layout

SOSU

문진표

처방전

릴스 처방전

처방

현재 릴스의 가장 큰 이탈 원인과 개선 방향을 짧게 요약합니다.

다음 릴스 적용

① 첫 장면 결과물 ② 짧은 첫 자막 ③ 설명보다 결과 우선 ④ A/B 테스트

톤앤매너

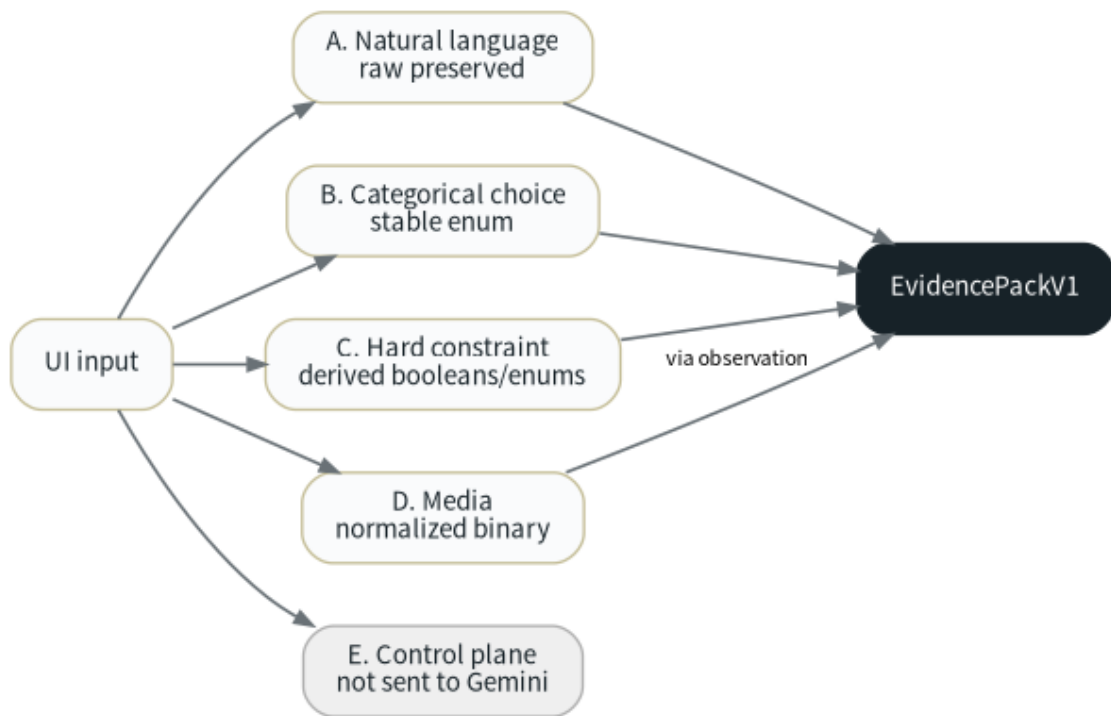
다음 릴스에서 유지해야 할 화면 분위기와 편집 리듬을 제시합니다.

처방전 받기

Prescription Report · fixed section layout

결과 화면 구조

3. AI에 들어가는 input의 다섯 분류



입력 분류

A. Natural-language evidence - 원문 보존형

Q1, Q2, Q3, Q4, Q5, Q7, Q8, Q12, Q13, Q14, Q15, Q20.

Python이 수행하는 것은 의미 해석이 아니라 정규화다.

- leading/trailing whitespace 제거
- 연속 공백 정리
- 빈 문자열은 `null`
- 최대 입력 길이 validation
- Unicode normalization
- 사용자 표현의 내용은 재작성하지 않음

예를 들어 Q7에 다음이 들어와도 그대로 유지한다.

다른 데는 보통 파스텔 조합만 하는데 저는 일부러 투명한 거랑 탁한 색을 섞고,
같은 조합도 조금씩 다르게 만들어요. 완전 똑같은 건 거의 없어요.

Python은 이를 희소성, 색감, 수공예성으로 요약해서 버리지 않는다. 단지 `brand_facts.differentiation_raw`라는 의미 슬롯에 배치한다.

B. Categorical evidence - stable code 변환형

Q6, Q9, Q10, Q11, Q16, Q17.

사용자에게는 한글 label을 보여주지만 request에는 stable code를 보낸다. 이 값은 Python이 추론하지 않는다. 계약에 정의된 enum인지 검증하고 semantic namespace에 배치한다.

예:

```
{
  "q6": ["design", "detail", "rarity"],
  "q9": ["aesthetic", "unique_ownership"]
}
```

C. Hard constraint – 선택값에서 deterministic 파생

Q18은 일반적인 취향 feature가 아니라 추천이 위반하면 안 되는 제작자 제약이다.

예:

```
{
  "exposure_mode": "face_only"
}
```

Python 파생:

```
{
  "raw_choice": "face_only",
  "face_allowed": true,
  "voice_allowed": false
}
```

이 값은 reel_structure, reel_type_mix, first_show_priorities의 constraint로 사용한다.

D. Media evidence – binary normalization 후 AI 관측

제품 사진이나 Insight screenshot은 Python이 의미를 rule-based로 추출하지 않는다.

- 브라우저: 크기/용량 1차 축소
- Python/Pillow: EXIF orientation, RGB, metadata 제거, 필요 시 재압축
- Gemini observation call: 이미지에서 실제 보이는 사실만 structured JSON으로 반환

E. Control-plane input – AI에 절대 보내지 않음

다음 값은 앱 orchestration용이다.

- 다음
- 진단
- 처방
- 진단서 받기
- 처방전 받기
- 이메일 주소
- 첫 화면으로 돌아가기
- 상단 문진표 | 처방전 탭
- loading/retry button

이메일이나 navigation state가 Gemini prompt에 섞이면 안 된다.

4. Q1-Q20 입력 계약 - 화면부터 EvidencePack까지

Q19는 현재 source에 존재하지 않으므로 API field도 만들지 않는다.

Q	API field	UI control	사용자 입력 예	Python 처리	EvidencePack 위치	주요 결과 영향
Q1	brand_name	text	모노유리	원문 보존	brand_facts.name	브랜드 요약
Q2	product_category	text	유리 비즈 키링과 작은 오브제	원문 보존	brand_facts.product_category_raw	포지셔닝, 타깃
Q3	hero_product	text	빛에 따라 색이 달라 보이는 유리 키링	원문 보존	brand_facts.hero_product_raw	대표 제품, first show
Q4	primary_material	text	체코 유리비즈, 스테인리스	원문 보존	brand_facts.material_raw	요약, 강점
Q5	material_reason	text	빛을 받으면 표정이 달라져서	원문 보존	brand_facts.material_reason_raw	강점, 촬영 포인트
Q6	appeal_codes	multi choice (<=3)	디자인/디테일/희소성	enum 검증	declared_identity.appeals	강점, 포지셔닝, first show
Q7	differentiation	text	매번 색 조합을 조금씩 다르게 만듭니다	원문 보존	brand_facts.differentiation_raw	포지셔닝, 차별성
Q8	target_customer	text	독특한 소품을 좋아하는 20~30대	원문 보존	customer_context.target_raw	메인 타깃
Q9	purchase_motive_codes	choice[]	예뻐서/나만의 것	enum 검증	customer_context.purchase_motives	타깃, 메시지, hook
Q10	desired_emotion_codes	multi choice (<=3)	설렘/특별함	enum 검증	customer_context.desired_emotions	tone, pacing
Q11	brand_trait_codes	multi choice (<=3)	따뜻함/특별함	별도 namespace	declared_identity.brand_traits	tone, copy, font direction
Q12	avoid_image_text	text	유아적이거나 값싸 보이는 느낌	원문 보존	declared_identity.avoidance_raw	금지조건, tone
Q13	brand_motif	text	별빛, 물결	원문 보존	declared_identity.motif_raw	모티프, 연출

Q	API field	UI control	사용자 입력 예	Python 처리	EvidencePack 위치	주요 결과 영향
Q14	primary_selling_point	text	빛에 따라 유리 색이 변하는 것	원문 보존	reels_context.primary_message_raw	first show, structure
Q15	interesting_making_process	text	비즈를 고르고 하나씩 연결하는 과정	원문 보존	reels_context.process_raw	reel type, body scene
Q16	instagram_pain_codes	multi choice (<=3)	조희수/아이디어	enum 검증	reels_context.pain_points	전략 처방, hook
Q17	current_reel_format_codes	multi choice (<=3)	완성품 위주	enum 검증	reels_context.current_formats	current-to-recommended gap
Q18	exposure_mode	single enum	얼굴만	booleans 파생	reels_context.creator_constraint	hard constraint
Q20	must_show	text	포장 전 햇빛에 비춰보는 장면	원문 보존	reels_context.must_show_raw	first show, structure

4.1 Q10과 Q11은 같은 label을 써도 같은 feature가 아니다

특별함을 예로 들면:

Q10 특별함 -> desired_emotions.specialness
 Q11 특별함 -> brand_traits.specialness

Q10은 고객이 느끼길 바라는 감정, Q11은 브랜드가 스스로 갖고 싶은 속성이므로 inference namespace를 분리한다.

4.2 기타 처리

other가 선택된 질문은 companion free-text field를 둔다.

```
{
  "appeal_codes": ["design", "other"],
  "appeal_other_text": "작품마다 미세하게 다른 손맛"
}
```

other_text는 stable code로 재해석하지 않고 raw evidence로 함께 보낸다.

5. Stable code dictionary

Q6 appeal_codes

code	UI label
design	디자인

code	UI label
color	색감
detail	디테일
texture	질감
material	소재
making_process	제작 과정
story	스토리
rarity	희소성
other	기타

Q9 purchase_motive_codes

code	UI label
aesthetic	예뻐서
special	특별해서
self_reward	나를 위해
gift	선물하려고
memory	추억을 간직하려고
space_styling	공간을 꾸미려고
unique_ownership	나만의 것을 갖고 싶어서

Q10 desired_emotion_codes / Q11 brand_trait_codes

두 질문은 code vocabulary는 같아도 field namespace가 다르다.

code	UI label
excitement	설렘
happiness	행복
warmth	따뜻함
healing	힐링
fun	재미
specialness	특별함
being_moved	감동
comfort	위로
other	기타

Q16 instagram_pain_codes

code	UI label
low_views	조회수가 안 나와요
slow_follower_growth	팔로워가 안 늘어요
content_ideation	어떤 콘텐츠를 만들지 모르겠어요
filming	촬영이 어려워요
editing	편집이 어려워요
posting_consistency	꾸준히 올리기 어려워요
low_purchase_conversion	구매로 연결되지 않아요
other	기타

Q17 current_reel_format_codes

code	UI label
rarely_posts	거의 안 올려요
finished_product	완성품 위주
making_process	제작 과정 위주
photo_video_mix	사진/영상 혼합
trend_reels	트렌드 릴스
vlog	브이로그

Q18 exposure_mode

code	UI label	derived constraint
face_and_voice	네	face=true, voice=true
neither	아니요	face=false, voice=false
face_only	얼굴만	face=true, voice=false
voice_only	목소리만	face=false, voice=true

6. Transport request와 내부 EvidencePack은 같은 객체가 아니다

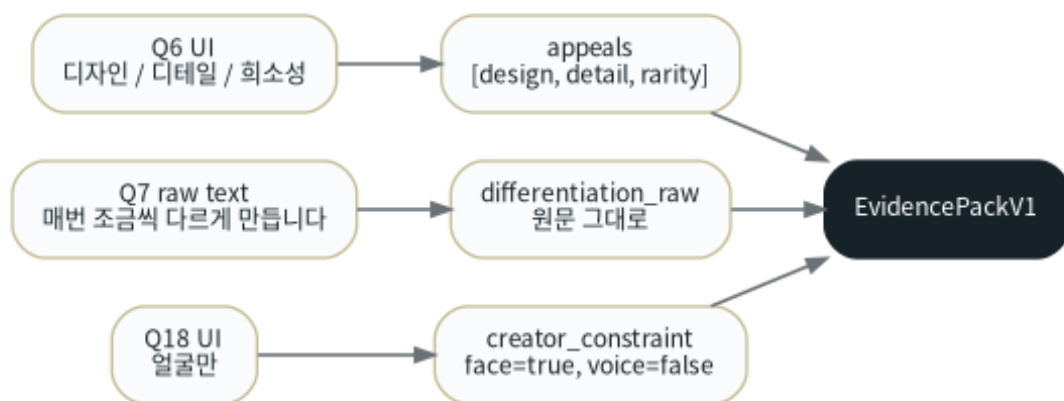
프론트는 화면에 가까운 request를 보내고, Python은 AI 추론에 적합한 domain object로 재구성한다.

6.1 Frontend -> AI service request 예

```
{
  "schema_version": "survey-submission.v1",
  "request_id": "diag_demo_001",
  "answers": {
    "brand_name": "모노유리",
    "product_category": "유리 비즈로 만드는 키링과 작은 오브제",
    "hero_product": "빛을 받으면 색이 달라 보이는 유리 키링",
    "primary_material": "체코 유리비즈",
    "material_reason": "빛에 따라 표정이 달라져서",
    "appeal_codes": ["design", "detail", "rarity"],
    "differentiation": "같은 색 조합으로 대량 제작하지 않고 매번 조금씩 다르게 만듭니다.",
    "target_customer": "흔한 캐릭터 제품보다 조금 독특한 소품을 좋아하는 20~30대",
    "purchase_motive_codes": ["aesthetic", "unique_ownership"],
    "desired_emotion_codes": ["excitement", "specialness"],
    "brand_trait_codes": ["warmth", "specialness"],
    "avoid_image_text": "너무 유아적이거나 값싸 보이는 느낌",
    "brand_motif": "별빛, 물결",
    "primary_selling_point": "빛을 받았을 때 유리가 반짝이는 것",
    "interesting_making_process": "색 조합을 고르고 하나씩 연결하는 과정",
    "instagram_pain_codes": ["low_views", "content_ideation"],
    "current_reel_format_codes": ["finished_product"],
    "exposure_mode": "face_only",
    "must_show": "포장하기 전에 햇빛에 비춰보는 장면"
  }
}
```

제품 이미지는 같은 multipart request의 `product_images[]` binary part로 전달한다. 이미지가 없으면 해당 part가 없다.

6.2 Python deterministic normalization



변환 예

Python은 위 request를 다음처럼 재배치한다.

```
{
  "brand_facts": {
    "name": "모노유리",
    "product_category_raw": "유리 비즈로 만드는 키링과 작은 오브제",
    "hero_product_raw": "빛을 받으면 색이 달라 보이는 유리 키링",
    "material_raw": "체코 유리비즈",
    "material_reason_raw": "빛에 따라 표정이 달라져서",
    "differentiation_raw": "같은 색 조합으로 대량 제작하지 않고 매번 조금씩 다르게 만듭니다."
  },
  "declared_identity": {
    "appeals": ["design", "detail", "rarity"],
    "brand_traits": ["warmth", "specialness"],
    "avoidance_raw": "너무 유아적이거나 값싸 보이는 느낌",
    "motif_raw": "별빛, 물결"
  },
  "customer_context": {
    "target_raw": "흔한 캐릭터 제품보다 조금 독특한 소품을 좋아하는 20~30대",
    "purchase_motives": ["aesthetic", "unique_ownership"],
    "desired_emotions": ["excitement", "specialness"]
  },
  "reels_context": {
    "primary_message_raw": "빛을 받았을 때 유리가 반짝이는 것",
    "process_raw": "색 조합을 고르고 하나씩 연결하는 과정",
    "pain_points": ["low_views", "content_ideation"],
    "current_formats": ["finished_product"],
    "creator_constraint": {
      "source": "face_only",
      "face_allowed": true,
      "voice_allowed": false
    },
    "must_show_raw": "포장하기 전에 햇빛에 비춰보는 장면"
  }
}
```

이 객체가 DeclaredEvidenceV1이다. 아직 이미지 의미는 들어오지 않는다.

7. 자연어 처리 원칙 - Python은 의미를 빼앗지 않는다

자연어 입력에 대해 backend에서 허용되는 deterministic transform은 다음뿐이다.

```
raw user string
-> Unicode normalize
-> trim
-> collapse accidental whitespace
-> empty => null
-> length validation
-> semantic slot placement
```

다음은 하지 않는다.

- keyword extraction으로 원문 대체
- sentiment label로 원문 대체
- LLM을 이용한 사전 요약 후 원문 폐기
- regex로 “희소성”, “고급”, “따뜻함” 같은 단어를 찾아 categorical feature로 승격
- 사용자가 쓰지 않은 사실 보완

필요하다면 최종 Gemini가 raw text를 읽고 의미를 해석한다. categorical 값은 이미 사용자가 명시적으로 눌렀기 때문에 별도로 안정된 prior로 존재한다.

8. 이미지 경로 - 저장소 없이 처리하는 기본형

기본 배포는 별도 GCS/Blob 저장소 없이 동작하도록 설계한다.

```
Original image
-> browser resize/compress
-> multipart/form-data
-> Python Pillow normalize
-> Gemini inline multimodal
-> request 종료 후 폐기
```

8.1 Browser 1차 전처리

목적은 Vercel/FastAPI ingress와 모델 비용을 동시에 줄이는 것이다.

- base64 JSON을 사용하지 않는다.
- multipart/form-data binary로 전송한다.
- 긴 변을 합리적 범위로 축소한다.
- JPEG/WebP 품질을 조정하되 작은 글자가 있는 Insight screenshot은 가독성을 우선한다.
- 최종 transport budget은 전체 multipart payload가 endpoint 한도를 넘지 않도록 프론트에서 검사한다.

8.2 Python 2차 normalization

media_kind에 따라 정책을 나눈다.

product_photo

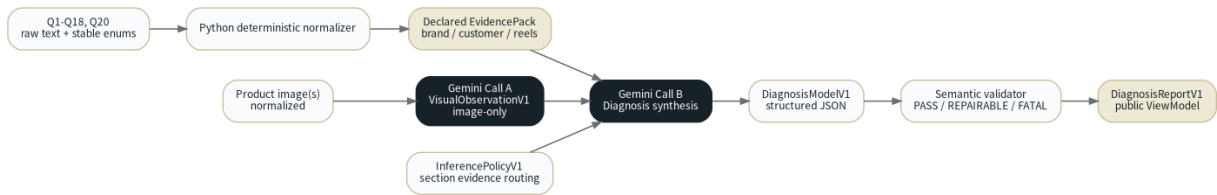
- EXIF rotate
- RGB conversion
- metadata strip
- 지나치게 큰 해상도 축소
- 시각적 texture/detail 보존

insight_screenshot

- 텍스트가 뭉개지지 않게 보수적 resize
- 과도한 JPEG artifact 금지
- OCR 대상 숫자/label contrast 보존

이미지 byte 자체를 장기 저장하지 않는 것이 기본값이다.

9. 메인 문진은 Gemini 2단 호출을 기본으로 한다



문진 AI 파이프라인

Call A - VisualObservationV1

제품 이미지가 있을 때만 수행한다.

중요: 이 call에는 Q1-Q20 문진 prior를 넣지 않는다. 모델이 사용자의 자기평가에 끌려 이미지에서 confirmation bias를 만드는 것을 줄이기 위함이다.

입력:

- normalized product image(s)
- neutral observation instruction
- observation JSON Schema

출력 예:

```
{
  "visible_product_type": "small_accessory",
  "dominant_visual_traits": [
    "transparent material",
    "fine bead detail",
    "mixed muted and clear colors"
  ],
  "material_cues": ["glass-like reflection"],
  "strongest_visual_signal": "light reflection",
  "possible_reel_assets": [
    "light movement",
    "macro detail",
    "hand assembly"
  ],
  "uncertain_observations": []
}
```

이 결과는 사실 관측이어야 하며 다음을 하지 않는다.

- 타겟 고객 추천
- 브랜드 포지셔닝 제안
- 사용자의 의도 추정
- 보이지 않는 소재/가격/브랜드 역사 창작

이미지가 없을 때

Call A를 생략한다. `visual_observation = null`로 `synthesis`를 수행한다. 결과는 문진 `prior`에 더 크게 의존하되, 이미지로 검증하지 못한 사실을 이미지 근거처럼 말하지 않게 `prompt`에서 제한한다.

10. EvidencePackV1 – 최종 synthesis 입력

Call B 직전에 Python이 다음 객체를 조립한다.

```
{
  "declared": {
    "brand_facts": {},
    "declared_identity": {},
    "customer_context": {},
    "reels_context": {}
  },
  "observed": {
    "visual_observation": {}
  },
  "constraints": {
    "creator": {
      "face_allowed": true,
      "voice_allowed": false
    }
  },
  "evidence_meta": {
    "has_product_image": true,
    "answered_fields": 18,
    "categorical_fields": 6,
    "free_text_fields": 12
  }
}
```

`evidence_meta`는 모델이 사용자에게 보여줄 score가 아니라 서비스 내부 trace다. 임의의 확률값을 만들지 않는다.

11. InferencePolicyV1 – “어텐션”을 section routing으로 통제

모델 내부 Transformer attention weight를 직접 설정하지 않는다. 대신 결과 section마다 어떤 source를 primary/supporting/constraint로 읽을지 명시한다.

결과 section	Primary evidence	Supporting evidence	Constraint / guard
Brand Summary	Q1, Q3, Q4	Q2, Q13	입력에 없는 사실 창작 금지
추천 포지셔닝	Q2, Q3, Q6, Q7	Q11, Q13, Visual	Q12 avoidance
핵심 강점	Q5, Q6, Q7, Q14, Q15	Q3, Visual	declared와 visual 불일치 시 구분
메인 타겟	Q8, Q9, Q10	Q2, Q3, Q11	demographic 과잉추정 금지
톤앤매너	Q10, Q11, Q12	Q13, Visual	Q18 제작자 제약 참고

결과 section	Primary evidence	Supporting evidence	Constraint / guard
추천 폰트	Q10, Q11, Q12	Q13, Visual	실제 라이선스 사실을 모델이 창작하지 않음
먼저 보여줄 것	Q14, Q15, Q20, Visual	Q6, Q9	Q18 hard constraint
릴스 유형 조합	Q16, Q17, Q18, Q20	Q6, Q9, Q15	Q18 hard constraint
30초 구조	Q14, Q15, Q16, Q17, Q20, Visual	Q6, Q9	Q18 hard constraint
최종 가이드	전체 synthesis	declared-vs-observed gap	실행 불가능 제안 금지

11.1 declared와 observed가 충돌할 때

예:

```
Q6 = rarity, design
VisualObservation = material reflection, fine detail
Visual에서 rarity 자체는 검증 불가
```

모델은 “희소성이 이미지에서도 확인된다”고 단정하지 않는다.

대신:

- declared strength: rarity
- observed strength: light reflection / fine detail
- synthesis: 희소성 주장은 제작과정/개별 조합을 보여줘야 설득력이 생김

처럼 gap을 recommendation으로 변환한다.

12. Call B – Diagnosis synthesis prompt composition

최종 request는 긴 자유형 문자열 하나가 아니라 논리적 block으로 구성한다.

SYSTEM

역할, 금지사항, evidence 사용 원칙

DECLARED EVIDENCE

EvidencePack.declared

OBSERVED EVIDENCE

VisualObservationV1 또는 null

CONSTRAINTS

creator_constraint 등 위반 금지 조건

INFERENCE POLICY

section별 primary/supporting/constraint routing

TASK

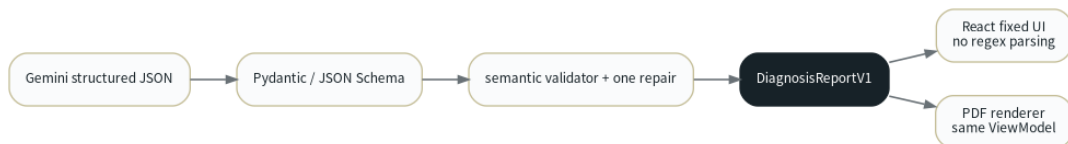
declared와 observed를 비교하여 실행 가능한 룰스 진단 생성

OUTPUT CONTRACT

DiagnosisModelV1 JSON Schema

Prompt 내부 구획은 XML/Markdown tag를 써도 된다. 그러나 모델 output을 XML tag pair로 받지 않는다. output은 JSON structured output만 사용한다.

13. 결과 계약 - UI 제목이 아니라 데이터만 반환



결과 바인딩

화면에 고정된 섹션 제목은 프론트 상수다. Gemini가 추천 포지셔닝, 핵심 강점 같은 UI heading을 생성하지 않는다.


```

{
  "schema_version": "diagnosis-report.v1",
  "request_id": "diag_demo_001",
  "status": "success",
  "data": {
    "brand_summary": {
      "name": "모노유리",
      "material": "체코 유리비즈",
      "representative_product": "유리 키링",
      "keywords": ["빛", "유리 디테일", "개별성"]
    },
    "positioning": {
      "title": "빛에 따라 표정이 달라지는 작은 유리 오브제",
      "body": "사용자는 디자인과 희소성을 강점으로 선언했고, 이미지에서는 유리 표면의 반사와 미세한 디테일이 가장 강한 시각 자산으로 관찰됩니다. 희소성은 완성품만 보여주기보다 조합이 달라지는 제작과정을 함께 보여줄 때 더 설득력 있게 전달됩니다."
    },
    "strengths": [
      {
        "title": "빛에 반응하는 소재감",
        "body": "정적인 완성품보다 움직임 속에서 유리 반사가 드러날 때 차별점이 선명해집니다."
      }
    ],
    "target": {
      "title": "개별성이 있는 작은 오브제를 찾는 20~30대",
      "body": "..."
    },
    "tone": {
      "keywords": ["따뜻한", "섬세한", "조용한 특별함"],
      "body": "..."
    },
    "fonts": {
      "headline": {
        "family_hint": "단정한 산세리프",
        "style_hint": "중간 이상의 굵기",
        "rationale": "..."
      },
      "body": {
        "family_hint": "가독성 높은 산세리프",
        "style_hint": "regular",
        "rationale": "..."
      }
    },
    "first_show_priorities": [
      {
        "rank": 1,
        "title": "햇빛을 통과하는 유리",
        "body": "첫 3초 안에 제품을 움직여 반사 변화를 보여줍니다."
      }
    ],
    "reel_type_mix": [
      {
        "rank": 1,

```

```
    "title": "제작 디테일형",
    "body": "현재 완성품 위주 콘텐츠에서 보이지 않던 제작 차별점을 보완합니다."
  },
],
"reel_structure": [
  {
    "slot": "0_3",
    "objective": "시각 hook",
    "scene": "햇빛에 제품을 움직여 색 변화 노출",
    "caption": "빛에 따라 달라지는 한 점"
  }
],
"final_guidance": {
  "summary": "...",
  "checklist": ["...", "..."]
}
}
```



메인/처방 결과 화면

13.2 강한 구조와 느슨한 의미의 경계

강하게 고정할 것

- JSON object shape
- 필수 section 존재

- rank type/order
- 30초 structure의 slot key
- enum
- nullability
- public error envelope

과도하게 고정하지 않을 것

- positioning 정확히 140자
- 모든 strength 정확히 3개
- 모든 문장을 동일 길이로 생성
- 특정 keyword를 반드시 포함

UI가 필요로 하는 구조는 strict하게, AI가 해야 하는 해석의 표현은 flexible하게 둔다.

14. Python validation – schema valid만으로 성공하지 않는다

Structured output이 JSON Schema를 만족해도 의미적으로 실패할 수 있다.

PASS

- 필수 section 존재
- 핵심 text가 빈 문자열이 아님
- rank/slot 중복 없음
- Q18 hard constraint 위반 없음
- placeholder (... , Lorem ipsum, N/A) 없음

REPAIRABLE

예:

- strengths=[]
- first_show_priorities 한 항목 누락
- 특정 section이 지나치게 추상적
- face_only인데 voice-over를 필수로 제안

처리:

```
원 결과 + validation errors
-> repair prompt 1회
-> 다시 schema + semantic validation
```

FATAL

- Gemini timeout
- JSON 자체 반환 실패
- repair 후에도 public contract 불충족
- image decode 실패

FATAL 결과를 status=success로 React에 보내지 않는다.

15. Token/length 정책

다음 수치는 모델 플랫폼의 최대치가 아니라 SOSU 서비스의 초기 generation budget이다.

Call	예상 visible output	초기 max_output_tokens	목적
Visual observation	500-1,200	2,048	이미지 사실 관측
Survey synthesis	2,000-3,500	8,192	전체 결과지 생성
Survey repair	필요한 section만	2,048-4,096	누락/위반 수정
Insight observation	500-1,200	2,048	screenshot metric/label 관측
Prescription synthesis	800-1,500	4,096	처방/액션/tone

문장 길이는 hard character cap보다 prompt guideline로 통제한다.

- positioning/target/tone: 2-4문장
- strength item: 1-3문장
- first-show priority: 한 줄 제목 + 1-2문장
- reel structure slot: objective/scene/caption을 짧은 실행 문장으로

운영 후 input_tokens, output_tokens, reasoning/thought usage, latency의 P50/P95를 보고 예산을 조정한다.

16. 처방전 AI 계약

처방전은 메인 문진과 별도 prompt contract를 사용한다.

16.1 Request

```
reel_url
+ insight_screenshot_1
+ insight_screenshot_2
+ insight_screenshot_3
```

reel_url은 v1에서 직접 분석 evidence가 아니다. 별도 Instagram fetcher/API가 없으므로 URL의 실제 콘텐츠를 모델이 봤다고 가정하면 안 된다.

16.2 Call A - InsightObservationV1

세 screenshot에서 다음을 structured observation으로 추출한다.

- visible metric name
- visible metric value
- visible period/label
- screen type
- unreadable regions

- observation confidence

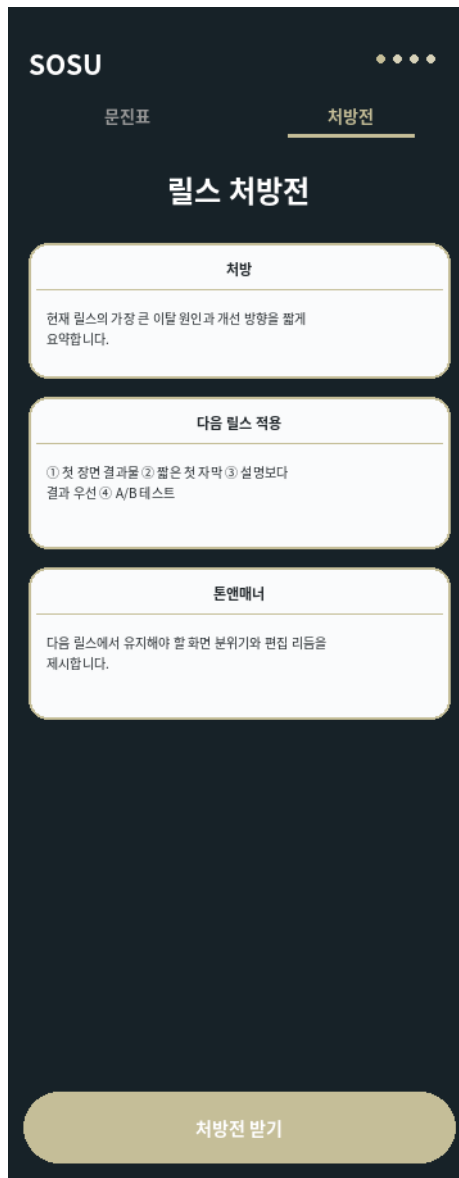
읽히지 않는 숫자를 추정하지 않는다.

16.3 Call B – Prescription synthesis

관측값을 기반으로:

```
{
  "prescription": {
    "title": "...",
    "body": "..."
  },
  "next_actions": [
    "...",
    "...",
    "...",
    "..."
  ],
  "tone": {
    "keywords": [...],
    "body": "..."
  }
}
```

를 반환한다.



처방전 결과 화면

17. Data plane과 Control plane을 분리한다

Data plane – AI 의미 생성에 사용

- Q1-Q18, Q20
- other_text
- product images
- insight screenshots
- prescription URL의 provenance string

Control plane – orchestration에만 사용

- button click
- current survey step
- active top tab

- loading state
- retry count at UI level
- email
- reset intent

특히 이메일 주소는 POST /v1/diagnosis / POST /v1/prescription request에 포함하지 않는다. 메일 전송 endpoint가 같은 DiagnosisReportV1 또는 PrescriptionReportV1을 받아 PDF/메일을 처리한다.

18. API surface

POST /v1/diagnosis

multipart

- payload: SurveySubmissionV1 JSON
- product_images[]: optional binary

200: DiagnosisReportV1

POST /v1/prescription

multipart

- payload: PrescriptionSubmissionV1 JSON
- insight_images[0..2]: exactly 3 binary

200: PrescriptionReportV1

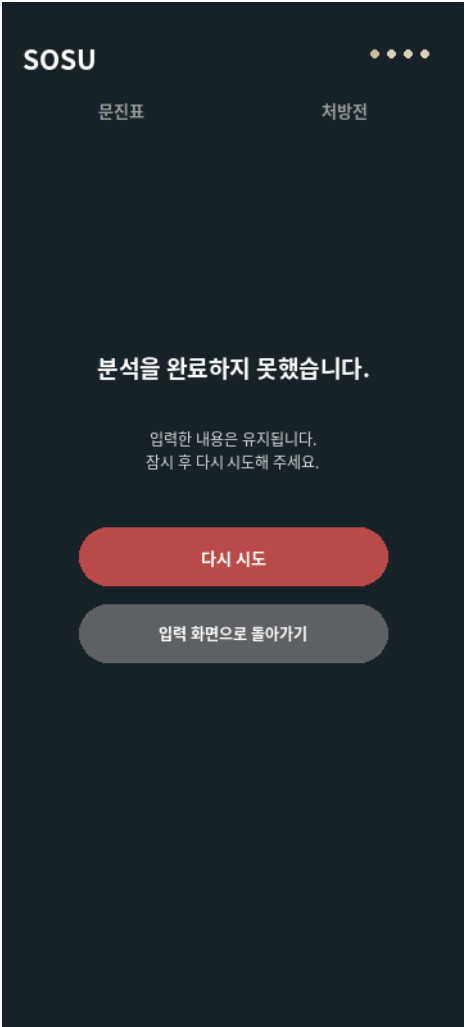
POST /v1/send-report

AI generation과 분리한다.

- email
- report type
- already validated public ViewModel 또는 server-side report id

Gemini를 다시 호출하지 않는다. 같은 ViewModel을 화면과 PDF가 공유한다.

19. Error contract



공통 오류 화면

code	의미	retry	프론트 처리
INPUT_INVALID	enum/type/field contract 불일치	No	해당 입력 표시
INPUT_INSUFFICIENT	의미 있는 답변도 이미지도 없음	No	문진으로 복귀
UNSUPPORTED_MEDIA	허용하지 않는 이미지 형식	No	업로드 영역 오류
PAYLOAD_TOO_LARGE	전송 예산 초과	No	브라우저 재압축/재선택
IMAGE_UNREADABLE	필요한 시각 evidence 판독 불가	Depends	이미지 교체 또는 survey는 이미지 없 이 재시도
MODEL_RATE_LIMIT	upstream 제한	Yes	입력 유지, 재시도
MODEL_TIMEOUT	모델 timeout	Yes	공통 retry
MODEL_UPSTREAM_ERROR	Gemini upstream 오류	Yes	공통 retry
MODEL_OUTPUT_INVALID	repair 후에도 public contract 불충족	Yes	공통 retry

React는 success 응답 내부에 부분 실패 section이 숨어 있지 않다고 가정할 수 있어야 한다.

20. Monorepo와 별도 에이전트 세션 경계

```
sosu/  
  apps/  
    web/  
      React application  
      Storybook  
  
  services/  
    ai/  
      app.py  
      routes/  
      domain/  
        submissions.py  
        evidence.py  
        observations.py  
        reports.py  
      normalizers/  
      prompts/  
        visual_observation/  
        survey_diagnosis/  
        insight_observation/  
        prescription/  
      adapters/  
        gemini.py  
        image.py  
      validators/  
      tests/  
  
  packages/  
    contracts/  
      openapi.yaml  
      survey-submission.v1.schema.json  
      prescription-submission.v1.schema.json  
      diagnosis-report.v1.schema.json  
      prescription-report.v1.schema.json  
      error.v1.schema.json  
      fixtures/  
  
  docs/  
    ssot/  
      UX_IA_SSOT.pdf  
      AI_SERVICE_SSOT.pdf
```

Frontend agent가 볼 것

- apps/web
- packages/contracts
- UX SSOT
- contract fixture

AI prompt/domain code를 열 필요가 없다.

AI agent가 볼 것

- services/ai
- packages/contracts
- 본 AI SSOT

React component 구현을 열 필요가 없다.

양쪽이 공유하는 것은 contract와 fixture뿐이다.

21. Storybook과 AI contract test의 연결

같은 fixture를 양쪽에서 사용한다.

Fixture	Frontend Storybook	AI test
survey_submission.valid	대표 입력 상태	normalizer/evidence 생성 검증
diagnosis_report.valid	Result 화면	public model serialization 검증
diagnosis_report.long_text	긴 결과 카드 QA	semantic max/budget QA
diagnosis_report.minimal_valid	최소 content UI	schema lower-bound test
prescription_submission.valid	처방 입력 mock	3장 media contract test
prescription_report.valid	처방전 결과	synthesis/public model test
error.model_timeout	Retry screen	error envelope test
error.output_invalid	Retry screen	repair exhausted test

이 방식이면 프론트가 실제 Gemini server가 없어도 모든 화면을 먼저 만들 수 있고, AI agent는 React를 보지 않고도 정확한 반환 구조를 맞출 수 있다.

22. End-to-end 예상 시나리오

단계 1 - 사용자가 화면에서 입력

Q6 디자인 / 디테일 / 회소성

Q7 같은 색 조합으로 대량 제작하지 않고 매번 조금씩 다르게 만들어요.

Q9 예뻐서 / 나만의 것을 갖고 싶어서

Q16 조회수가 안 나와요 / 어떤 콘텐츠를 만들지 모르겠어요

Q18 얼굴만

Q20 포장하기 전에 햇빛에 비춰보는 장면

+ 제품 사진 2장

단계 2 - Frontend transport

```
{
  "appeal_codes": ["design", "detail", "rarity"],
  "differentiation": "같은 색 조합으로 대량 제작하지 않고 매번 조금씩 다르게 만들어요.",
  "purchase_motive_codes": ["aesthetic", "unique_ownership"],
  "instagram_pain_codes": ["low_views", "content_ideation"],
  "exposure_mode": "face_only",
  "must_show": "포장하기 전에 햇빛에 비춰보는 장면"
}
```

단계 3 - Python normalizer

```
categorical labels -> stable enum 확인
natural language -> 원문 정리/보존
face_only -> face=true, voice=false
```

단계 4 - Visual Observation

모델은 문진 내용을 모른 채 이미지에서:

```
{
  "strongest_visual_signal": "light reflection",
  "possible_reel_assets": ["macro detail", "light movement"]
}
```

같은 관측을 만든다.

단계 5 - Synthesis

rarity는 declared evidence지만 이미지에서 직접 검증되지 않는다. 반면 light reflection은 observed evidence다.

따라서 모델은:

희소성 자체를 이미지가 증명한다고 말하지 않고, 사용자가 선언한 희소성을 매번 달라지는 조합/제작과정으로 보여주고, 시각 hook은 이미지에서 실제 강하게 관찰된 빛 반사를 이용한다.

라는 식으로 synthesis한다.

단계 6 - Public ViewModel

React는 위 reasoning을 파싱하지 않는다. 검증된 positioning, first_show_priorities, reel_structure만 받는다.

단계 7 - 화면/PDF

동일한 `DiagnosisReportV10`:

- React 결과 카드
- 메일용 PDF renderer

두 곳의 단일 content source가 된다.

23. 구현 acceptance gate

AI 서비스는 다음이 모두 통과해야 완료로 본다.

- ☐ 모든 categorical UI 값은 stable enum으로 검증된다.
- ☐ 자연어는 원문 의미를 보존하고 semantic slot에만 배치된다.
- ☐ Q10과 Q11은 별도 namespace다.
- ☐ Q18은 hard constraint로 파생된다.
- ☐ Q19 field는 존재하지 않는다.
- ☐ 제품 이미지가 있으면 문진 prior와 분리된 image-only observation이 먼저 수행된다.
- ☐ 이미지가 없으면 visual call을 생략하고 prior-only diagnosis가 가능하다.
- ☐ final synthesis는 section별 evidence routing을 사용한다.
- ☐ Gemini output은 JSON structured output이다.
- ☐ regex/XML output parsing이 없다.
- ☐ schema validation 뒤 semantic validator가 동작한다.
- ☐ repair는 최대 1회다.
- ☐ malformed partial result를 success로 반환하지 않는다.
- ☐ React는 public ViewModel만 렌더링한다.
- ☐ 이메일/버튼/navigation 값은 diagnosis prompt에 들어가지 않는다.
- ☐ 처방전 URL을 자동 fetch된 evidence로 오인하지 않는다.
- ☐ Insight screenshot에서 읽히지 않는 metric을 추정하지 않는다.
- ☐ browser compression -> Python normalization -> Gemini 경로가 외부 파일 저장소 없이 동작한다.
- ☐ frontend와 AI agent가 packages/contracts와 fixture만 공유해 독립 개발 가능하다.

24. 최종 데이터 계층 요약

```
UI
|
|-- free text -----> raw text preserved
|-- categorical choice -----> stable enum
|-- exposure mode -----> derived hard constraint
|-- product/insight image -----> normalized binary -> observation JSON
|-- buttons/email/navigation -----> control plane only
|
v
SurveySubmissionV1 / PrescriptionSubmissionV1
|
v
Deterministic Normalizer
|
v
DeclaredEvidenceV1
|
+---- VisualObservationV1 / InsightObservationV1
|
v
EvidencePackV1
|
v
InferencePolicyV1
|
v
Gemini structured synthesis
|
v
DiagnosisModelV1 / PrescriptionModelV1
|
v
Schema validation + semantic validation + one repair
|
v
DiagnosisReportV1 / PrescriptionReportV1
|
+---- React fixed UI
+---- deterministic PDF renderer
```

이 계층을 기준으로 프론트와 AI service는 서로의 내부 코드를 보지 않고도 독립적으로 구현한다.